

Universal Lesion Segmentation: Targeting class imbalance and input permutation robustness

O. Geertsema, D. Santos, V. Petre and H. Zhu
Supervisor: A. Hering

Abstract—Universal lesion segmentation can help radiologists reduce their workload and hopefully improve the percentage of accurate diagnoses. In this paper we adapt an existing state-of-the-art deep learning architecture (nnU-Net) to perform segmentation on the ULS’23 Challenge dataset, consisting of multiple different types of thorax-abdomen lesions. We test training protocols prioritizing different aspects of segmentation performance, and achieve the best results with models trained using a Dice loss term in combination with either a focal, top- k , or binary cross entropy loss term.

I. INTRODUCTION

The estimated number of cancer cases is expected to rise from 4.47 million in 2022 to 4.86 million in Europe alone [1]. This increased rate of diagnosis will be accompanied by an increased demand for medical imaging interpretation by radiologists. Although complete 3D segmentation of lesions from diagnostic imaging (often via computer tomography, CT) provides the best information for lesion monitoring over time, performing such segmentation manually is time-consuming, and prone to significant intra-reader variability [2]. Deep learning models have shown potential in automatic lesion segmentation, performing well on datasets limited to lesions in specific tissue types [3]. A logical next step is the development of models capable of segmenting lesions across a broad range of tissues.

To this end, we adapted an existing state-of-the-art biomedical image segmentation framework (nnU-Net, [3]) for universal lesion segmentation. We trained nnU-Net on the Universal Lesion Segmentation 2023 (ULS’23) Challenge dataset [4], consisting of lesions found across a wide variety of tissues in the thorax-abdomen area (bone, pancreas, lungs, etc.). Model performance on this challenge was evaluated using a weighted combination of 3 metrics, computed on a hold-out testing data split:

- 1) Average model segmentation performance as measured by the Dice coefficient (segmentation Dice).
- 2) Symmetric mean absolute percentage error (SMAPE) for the long- and short-axis measurements of the output segmentation.
- 3) Consistent model segmentation when the input volume of interest (VOI) is shifted in space (consistency Dice).

We first trained a baseline model using a standard combination of the Dice coefficient and binary cross entropy (BCE), and expanded upon this protocol using additional loss terms designed to target different facets of the ULS’23 model

evaluation metric. Specifically, we introduced additional loss terms encouraging the following, in separate experiments:

- 1) Generation of lesion segmentations with long- and short-axis dimension targets derived from the ground truth, targeting point 2).
- 2) Robustness of model outputs against rotated inputs, targeting point 3) without having to re-crop VOIs.
- 3) Focus of training on areas which are difficult to segment by building upon regular pixel-wise BCE with top- k and focal losses. These terms indirectly address the class imbalance problem between fore- and background pixels in the target masks. [5] [6].

In addition to the individual experiments above, we trained a model using a compound loss consisting of the Dice coefficient, and the aforementioned top- k and rotation-robustness loss terms, in order to better understand their combined effects on training. This combination was selected based on small-scale experimentation with each modification individually. Finally, due to part of the training data consisting of overlapping and poorly-segmented lesions, we attempted to use a watershed-based heuristic to further pre-process the target masks prior to training. We report increased ULS’23 leaderboard scores over the baseline when training with a top- k loss alone, though these differences were not statistically significant.

II. METHOD

A. Data

Training was performed on the ULS’23 challenge dataset [4]. The dataset consists of 3D VOIs of $256 \times 256 \times 128$ voxels obtained from CT scans, where the central pixel of each VOI is inside a lesion, and no lesion extends beyond the VOI. Each VOI contains a single lesion. The dataset contains 11 sub-datasets, covering various tissues in the thoracic-abdominal region, with binary target masks generated either through manual annotation or the Grabcut algorithm [7]. The latter data is of much poorer quality than the rest, and its inclusion was previously not found to drastically improve model performance under the inference time restrictions of the ULS’23 challenge [8]. Due to training time constraints, we therefore excluded this data from our training protocol. Furthermore, we sampled custom training and validation splits from each of the 11 sub-datasets, with $\sim 10\%$ and $\sim 2.5\%$ of the total sub-dataset contributing to each split, respectively

¹. All of a patient’s scans were assigned to one of either the training or validation split. Training was performed without cross-validation due to time limitations.

B. Baseline

We started with the baseline provided by the ULS’23 challenge [4], consisting of a standard nnU-Net [3] with minor extensions. nnU-Net is a self-configuring state-of-the-art deep learning segmentation method built on top of the standard U-Net architecture [9]. nnU-Net extracts a “fingerprint” from the training data, using this information to determine the optimal U-Net architecture for segmenting the specific dataset. The ULS’23 baseline makes two minor modifications to the standard training protocol. The first prevents the architecture from resampling training image resolutions based on the dataset fingerprint. This was done to cope with different sub-datasets containing images at different resolutions, as well as to accelerate inference. The next modification prevents nnU-Net from cropping into patches of the VOIs during training, keeping lesions in the center of each VOI. This is necessary, as the intended clinical use behind the ULS’23 challenge is for a radiologist to indicate a lesion voxel in a CT scan, after which a VOI is cropped around said point and segmentation is performed. This baseline was trained using a loss function consisting of the Dice coefficient and BCE, using a weighting scheme of 1:1 for the two terms.

All models in this study were trained from scratch using the 3D full-resolution segmentation configuration, using the default nnU-Net training procedures unless otherwise specified. All models were defined using the “M”-sized residual encoder preset, as residual encoders have been shown to boost segmentation performance [10]. All models were trained over 500 epochs, with an initial learning rate of 10^{-2} .

C. Modifications

1) **Long- and short-axis matching:** Although training using Dice and BCE is expected to lead to models segmenting lesions such that they have similar axis measurements to the ground truth, this facet is not explicitly targeted by the standard loss function. Furthermore, a lesion’s long- and short-axis sizes are relevant for assessing tumor development [11], motivating the addition of loss term that explicitly encourages axis size correspondence.

Long- and short-axis dimensions were extracted from the target and output masks by fitting an ellipse to each 2D slice within a VOI along the z-axis. The SMAPE metric was computed across all slices of all VOIs within a training batch, leading to a loss term bounded between 0 and 1 (0 = identical axis measurements between target and output). By including this term in the loss, deviations from target axis lengths are explicitly penalized. The final loss function for this experiment used a weighting scheme of 0.4, 0.4, 0.2 for Dice, BCE, and SMAPE loss terms, respectively.

¹This excludes the smaller MDSC-Lung and MDSC-Colon datasets from which all images were included, and partitioned roughly 80%/20% towards training and validation splits, respectively.

2) **Robustness against rotation:** As the ULS’23 challenge tests model robustness to shifting the input VOIs in 3D space, we extended the default loss function to address this. For each forward pass during training, we copy the current batch of input VOIs, rotate it 180 degrees around the z-axis, concatenate it with the original batch, and perform the forward pass. We then un-rotate the outputs corresponding to the rotated inputs, and compute the absolute voxel-wise differences between the outputs corresponding to the original and rotated inputs. We average these differences across all voxels in each output segmentation map, and then average this across the batch to generate a single rotation-robustness loss term. This term therefore describes the average difference in logit values across output masks, when the input is rotated 180 degrees. The final loss function for this experiment used a weighting scheme of 0.4, 0.4, 0.2 for Dice, BCE, and rotation-robustness loss terms, respectively.

We enforced rotation-robustness hoping that it indirectly led to robustness against translating input VOIs in 3D space. We opted for this approach instead of directly translating VOIs in space during training, as the latter would require re-pre-processing of the ULS’23 dataset to generate the necessary shifted VOIs, in addition to modifications grouping the shifted VOIs together during training. We only rotated the input 180 degrees due to GPU memory limitations.

3) **Top- k and focal loss functions:** The ULS’23 dataset suffers from large class imbalance, where the majority of target voxels belong to the background. As a result, most of the training is focused on correctly identifying background voxels, giving crucial areas such as the foreground-background boundary much less weight. To address this imbalance, we replace the BCE term in the default loss function by top- k loss [5] and focal loss terms [6] respectively.

Top- k loss computes voxel-wise BCE between the output and target masks, and only selects the top $k\%$ largest resulting error terms within each VOI to contribute to the final loss term. Focal loss is a softer version of this, instead modulating the weight that each voxel-wise error contributes to the final loss term via a tunable parameter γ , again giving the larger errors larger weight during the training process. In both cases, this focuses training on difficult-to-classify voxels, discouraging models from minimizing losses by over-predicting background labels. Training segmentation models using both top- k and focal loss variants of CE together with Dice loss has been shown to perform consistently better than other loss functions [5]. We tested each approach individually.

We used the existing nnU-Net implementation of the top- k loss term with a value of $k = 10$. We used the Kornia package [12] for the focal loss term, keeping its default values with $\gamma = 2$. Both loss functions used a 1:1 weighting scheme between the top- k /focal loss term and Dice.

4) **Watershed segmentation:** Some labels within one of the sub-datasets (the LiTS dataset) actually contain multiple lesions close together [8]. Noting that most individual lesions tend to be small and somewhat circular, Watershed segmentation [13] was implemented with the aim of correcting such

labeling errors, to improve dataset quality.

The implementation scans through all 2D slices of a given VOI along the z-axis, identifying those for which the lesion area is above a certain size and below a circularity threshold. Circularity of lesions is calculated using the metric $C = 4\pi * \frac{\text{area}}{\text{perimeter}^2}$. Once identified, a standard Watershed segmentation algorithm is applied on these slices, followed by further post-processing to remove the voxels where the individually-segmented components touch. The central voxel of the resulting 3D VOI mask is then used as a seed point for connected components analysis in 3D, after which all label voxels not belonging to the volume containing the central voxel are removed. In doing this, we attempt to separate the multiple lesions within the mask (should they exist), and keep only the one in the center of the VOI.

After processing the LiTS dataset in this way, we intended to train another model from scratch on the complete dataset and test for performance differences versus the baseline. Unfortunately, there were many cases where the Watershed segmentation yielded inconsistent results across slices of the same lesion, splitting the lesion label in one slice but not doing so for an adjacent slice. Connected components analysis was therefore often unable to separate the components in 3D, despite testing a range of hyperparameter values. Given that we only trained on 3D data, we abandoned this method.

D. Experiments

We trained a total of 6 models on the custom training split defined in Section II-A. We trained the baseline as specified in Section II-B, and trained one additional model for each of the custom loss functions discussed in Section II-C (axis-matching, rotation-robustness, top- k , focal). Finally, we trained a model using both top- k and rotation-robustness terms, in addition to the standard Dice term. We selected this combination as top- k and robustness terms individually showed the most promise in our initial experiments (not reported here). We performed internal validation during training by computing pseudo Dice scores on the validation split defined in Section II-A, and tested our models by submitting them to the ULS’23 development leaderboard. Code for training models and submitting to Grand Challenge is available at https://github.com/haozez/aimi_uls_final.

III. RESULTS

All models trained successfully, though none managed to reach convergence within the allocated 500 epochs (see Figure 1). Furthermore, there was a striking difference in the training progression across the different loss functions. In particular, loss functions including the rotation-robustness term showed a sudden phase transition around epoch ≈ 160 . Prior to this point, the loss curves were also completely flat and showed no indication of eventual improvement (not shown in the interest of space). Although direct model comparison is difficult due to the lack of convergence, we compared performance on the ULS’23 development leaderboard split across each of the 4 metrics contributing to each model’s final score. We compared

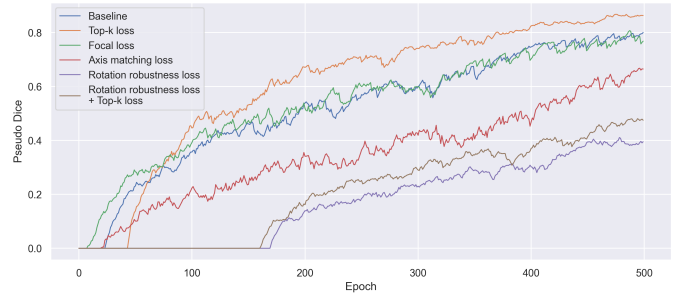


Fig. 1: Pseudo Dice scores on validation splits across training iterations, for all 6 models. Top- k model consistently outperformed all other models after ≈ 100 iterations. Losses including a rotation-robustness term caused a dramatic phase transition at epoch ≈ 160 , which was also reflected in the training losses for the respective models.

Model	Top- k	Baseline	Focal
Final score	0.70	0.69	0.69
Consistency Dice	0.80 ± 0.24	0.74 ± 0.32	0.78 ± 0.29
Segmentation Dice	0.66 ± 0.25	0.66 ± 0.27	0.65 ± 0.26
Long axis percentage error	0.12 ± 0.14	0.12 ± 0.19	0.12 ± 0.18
Short axis percentage error	0.15 ± 0.18	0.15 ± 0.21	0.14 ± 0.21

Model	Axis-matching	Rotation-robustness	Combined
Final score	0.64	0.62	0.61
Consistency Dice	0.73 ± 0.29	0.67 ± 0.29	0.66 ± 0.34
Segmentation Dice	0.61 ± 0.27	0.59 ± 0.25	0.57 ± 0.28
Long axis percentage error	0.17 ± 0.20	0.14 ± 0.13	0.16 ± 0.18
Short axis percentage error	0.17 ± 0.23	0.15 ± 0.17	0.18 ± 0.21

Table 1: Scores on the ULS’23 Challenge development leaderboard. Final score is a weighted average of the 4 other metrics below it. Results are reported as mean $\pm$ standard deviation. “Combined” refers to a model trained using Dice, top- k , and rotation-robustness loss terms simultaneously. **Bold** indicates statistically significant differences in performance versus the baseline, for each individual metric ($p < 0.05$).

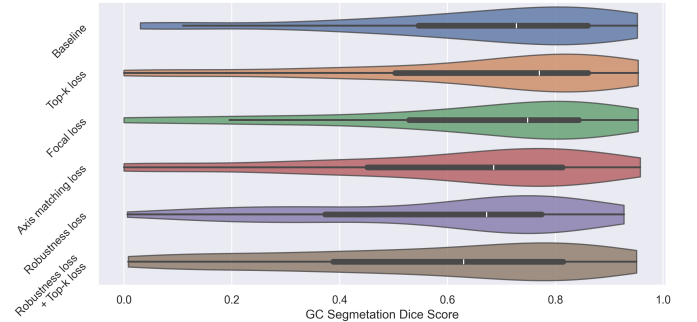


Fig. 2: Visualization of segmentation Dice distributions on the ULS’23 development leaderboard dataset, across all 6 models.

each model’s metric against the baseline model’s metric using a two-tailed Wilcoxon signed-rank test, finding significant differences between the performance of the baseline versus models trained with axis-matching, rotation-robustness, and rotation-robustness + top- k (see Table 1). All significant differences were indicative of poorer model performance versus the baseline. Although the model trained with top- k alone led to a larger consistency Dice score and a larger final score, this difference was not statistically significant. Moreover, this

model did not universally produce superior segmentations compared to the other models on the development leaderboard.

As seen in Figure 2, the poorer performance of the rotation-robustness and rotation-robustness + top- k -trained models manifests itself not only through a leftward shift of the distributions of segmentation Dice scores, but also through a widening of the left distribution tail. Analyzing model outputs, this appears to be a result of the rotation-robustness term prioritizing the production of masks symmetrical against rotation about the z-axis, instead of focusing on correctly segmenting lesions (see Figure 3).

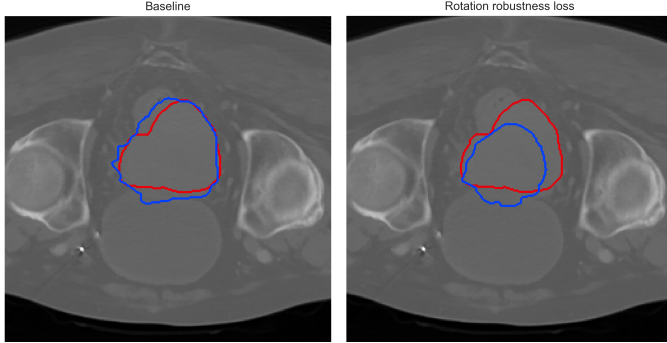


Fig. 3: Output produced by baseline and rotation-robustness-trained models. Red = ground truth, blue = model output. Note how the rotation-robust model produces a mask highly-symmetrical in terms of rotation about the z-axis, under-segmenting much of the lesion. Similar behavior was seen in the top- k + rotation-robust model, though to a lesser extent.

IV. DISCUSSION & CONCLUSION

The primary approach for our modifications was to improve model performance under the specific facets targeted by the ULS’23 challenge. Unfortunately, many modifications had the opposite effect. Training with an axis-matching loss term led to significantly poorer SMAPE scores than the baseline (higher = worse), and training with a rotation-robustness loss term led to a significantly lower consistency Dice score. In terms of axis-matching, this is possibly due to poor method robustness. Under circumstances where slices only had a few lesion voxels labeled, fitting an ellipse was impossible, and target values of 0 were used instead. Better practice would be to empirically determine the target values to use for these specific cases. Rotation-robustness seems to degrade performance by providing the model with an easy way of minimizing the loss (generating rotation-symmetrical segmentation masks, see Figure 3), taking focus away from generating accurate segmentations. Although it is possible that further training would encourage the model away from this strategy, it is likely that the weight assigned to the rotation-robustness loss term was too high, as it was set based on intuition as opposed to empirical evidence. In addition, enforcing model robustness via rotation is in itself probably not the best practice, as it only indirectly targets the desired behavior (robustness to VOI translation) in the best case scenario. Modification

of the training dataset to explicitly include the same VOIs translated in space, with explicit penalization of differing model outputs across these translated inputs during training, is a promising direction for future research. A significant caveat to this discussion is that models are being compared prior to convergence, on a very small dataset; it may well be that the potential of the different training methods relative to each other is not accurately represented after only 500 epochs.

Aside from differences in performance across the training approaches, the behavior of the models during training is strikingly different. All models trained with a rotation-robustness loss term show no signs of generalization until over 100 epochs into the training process (see Figure 1). We offer two speculative explanations for this phenomenon. The first is that the rotation-robustness loss term with our chosen weight drastically alters the loss landscape, such that the optimizer spends a long time traversing a flat surface. Although we incorporate momentum into our optimizer to avoid getting stuck in such regions of the space [14], we only attempted training with stochastic gradient descent. It is possible that using a more sophisticated optimizer such as Adam would avoid this issue. A second possible explanation is that it is not the rotation-robustness term alone contributing to such abnormal training behavior, but rather a combination of the rotation-robustness term with a dataset that is too small. “Grokking” describes a phenomenon where a network trained on a small dataset will only start showing generalization to a validation dataset after long periods of overfitting to the training dataset [15]. Although this phenomenon has primarily been shown with synthetic datasets and over-parameterized networks, it is possible that similar underlying conditions result when training with the rotation-robustness loss term and our custom dataset splits. In this sense, future experiments with larger dataset and varying rotation-robustness term weights could potentially reveal further insight into this behavior.

In conclusion, we have investigated different approaches to training deep learning architectures for universal lesion segmentation. We trained nnU-Net architectures on a diverse dataset of lesions across numerous thorax-abdomen tissues, using loss functions designed to target different aspects of segmentation performance. Through our experiments, we find that both training the models to generate segmentations with specific lesion axis measurements, as well as training models to be robust against input rotation, leads to significantly worse performance than the baseline. In addition, all our experiments training with rotation-robustness loss terms led to no model generalization on unseen data for over 100 epochs, after which a striking phase transition occurred. Finally, we tested the use of top- k /focal loss terms in place of binary cross-entropy during training and find no significant performance differences versus the baseline model, though we do report a higher score than the baseline on the ULS’23 development leaderboard when training with a combination of top- k and Dice losses. Although initially promising, our results are severely limited by the small dataset size, and the lack of model convergence during the available training time.

REFERENCES

- [1] Globocan, “Cancer tomorrow: Estimated number of new cases from 2022 to 2030, both sexes, age [0-85+]”, Global Cancer Observatory
- [2] Tang, Y., Harrison, A. P., Bagheri, M., Xiao, J., & Summers, R. M. (2018). Semi-automatic recist labeling on CT scans with cascaded convolutional neural networks. MICCAI 2018, 405–413. https://doi.org/10.1007/978-3-030-00937-3_47
- [3] F. Isensee, P.F. Jaeger, S.A.A. Kohl, J. Petersen and K.H. Maier-Hein, “nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation”, Nat Methods, p. 203-211, <https://doi.org/10.1038/s41592-020-01008-z>
- [4] M. de Grauw, B. van Ginneken and A. Hering, “The universal lesion segmentation ‘23 challenge”, Diagnostic Image Analysis Group, <https://uls23.grand-challenge.org/uls23/>
- [5] Ma, J., Chen, J., Ng, M., Huang, R., Li, Y., Li, C., Yang, X., & Martel, A. L. (2021). “Loss odyssey in medical image segmentation”, Medical Image Analysis, 71, 102035. <https://doi.org/10.1016/j.media.2021.102035>
- [6] Lin, T.-Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). “Focal loss for dense object detection”, 2017 IEEE ICCV. <https://doi.org/10.1109/iccv.2017.324>
- [7] T. Urban, E. Ziegler, S. Pieper, J. Kirby, D. Rukas, B. Beardmore, et al, “Crowds Cure Cancer: Crowdsourced data collected at the RSNA 2018 annual meeting [Data set].”, The Cancer Imaging Archive, 2019, <https://doi.org/10.7937/TCIA.2019.yk0gm1eb>
- [8] M.J.J. de Grauw, E. T. Scholten, E. J. Smit, B. van Ginneken, A. Hering. “ULS23: Benchmark and Baseline for 3D Universal Lesion Segmentation in Computed Tomography” (To be submitted)
- [9] O. Ronneberger, P. Fischer and T. Brox, “U-Net: convolutional networks for biomedical image segmentation”, CoRR, 2015, <http://arxiv.org/abs/1505.04597>
- [10] Isensee, F., Wald, T., Ulrich, C., Baumgartner, M., Roy, S., Maier-Hein, K., & Jäger, P. (2024). “nnU-Net Revisited: A call for rigorous validation in 3D medical image segmentation”, arXiv:2404.09556
- [11] “Radiology Assistant”, <https://radiologyassistant.nl/more/recist-1-1/recist-1-1>
- [12] Riba, E., Mishkin, D., Ponsa, D., Rublee, E., & Bradski, G. (2020). “Kornia: An open source differentiable computer vision library for PyTorch”, 2020 IEEE WACV. <https://doi.org/10.1109/wacv45572.2020.9093363>
- [13] Beucher, S., & Meyer, F. (2018). “The morphological approach to segmentation: The Watershed Transformation”, Mathematical Morphology in Image Processing, 433–481. <https://doi.org/10.1201/9781482277234-12>
- [14] Sutskever, I., Martens, J., Dahl, G. & Hinton, G. (2013). “On the importance of initialization and momentum in deep learning”, Proceedings of the 30th International Conference on Machine Learning 28(3):1139-1147
- [15] Power, A., Burda, Y., Edwards, H., Babuschkin, I., & Misra, V. (2022). “Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets”, arXiv:2201.02177